

Steps to generate a Quantum Variational Circuit (VQC_BetheSalPeter).

This manual provides a step-by-step guide to creating a modular Variational Quantum Circuit, designed for regression and parameter optimization tasks.

1. Configuración del Entorno

Before you begin, make sure you have the necessary dependencies installed. We recommend using a Python virtual environment.

pip install pennylane numpy matplotlib

```
In [1]: #=====
# Librerias
#=====
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt
```

2. Circuit Architecture

The circuit follows a hardware-efficient architecture. This section defines the parameterized Ansatz.

```
In [5]: # Configuración del dispositivo
wires = 4
dev = qml.device("default.qubit", wires=wires)

def layer(weights):
    """Capa base: Rotaciones Eulerianas y Entrelazamiento CNOT"""
    for i in range(wires):
        qml.Rot(weights[i, 0], weights[i, 1], weights[i, 2], wires=i)
    for i in range(wires - 1):
        qml.CNOT(wires=[i, i + 1])
```

3. The Quantum Node (QNode)

Here we combine data encoding (Input) with the variational layers. This is the "engine" that will be optimized.

```
In [6]: @qml.qnode(dev)
def vqc_circuit(params, x):
    # 1. Feature Map: Mapeo de datos clásicos al espacio cuántico
    qml.AngleEmbedding(x, wires=range(wires))
    # 2. Ansatz: Capas parametrizadas
```

```

for p in params:
    layer(p)
# 3. Medición: Extracción del valor esperado
return qml.expval(qml.PauliZ(0))

```

4. Visualizing and Printing the Circuit

To include this circuit in your thesis or report, you can generate a visual representation of it.

```

In [10]: # Definición de parámetros para la visualización
layers = 2
params = np.random.uniform(0, np.pi, (layers, wires, 3))
x_test = np.random.random(wires)

```

```

# Dibujar el circuito
print(qml.draw(vqc_circuit)(params, x_test))

```

```

...
Note for printing: The command above will print an ASCII
text diagram to your console. For a high-quality version (LaTeX/PDF),
you can use: `fig, ax = qml.draw_mpl(vqc_circuit)(params, x_test)`
and save it as a .pdf or .png file.
...

```

```

0: -rAngleEmbedding(M0)---Rot(2.45,1.52,0.93)-r•---Rot(0.75,2.58,1.6
1)-----
1: -|AngleEmbedding(M0)---Rot(2.22,0.26,2.13)-\X-r•-----Rot(0.26,1.6
2,1.73)
2: -|AngleEmbedding(M0)---Rot(0.60,1.24,2.71)-----\X
-----r•-----
3: -\AngleEmbedding(M0)---Rot(1.27,0.65,1.74)-----\X
-----

```

```

-----r•-----| <Z>
-----\X-----r•-----|
-----Rot(1.02,1.91,2.37)-\X-r•-----|
-----Rot(2.44,1.78,0.43)-----\X-----|

```

```

M0 =
[0.11907497 0.95910555 0.59311484 0.68669531]

```

```

Out[10]: '\nNote for printing: The command above will print an ASCII \ntext diagram to your
console. For a high-quality version (LaTeX/PDF),\nyou can use: `fig, ax = qml.draw
_mpl(vqc_circuit)(params, x_test)\n` and save it as a .pdf or .png file.\n'

```

5. Circuit Training

Finally, a simple routine to optimize the parameters using a classic optimizer (Adam).

```

In [9]: opt = qml.AdamOptimizer(stepsize=0.1)

def cost(params):
    return np.abs(vqc_circuit(params, x_test) - 0.5)**2 # Ejemplo: ajustar a 0.5

for i in range(20):

```

```
params = opt.step(cost, params)
print(f"Iteración {i}: Coste = {cost(params):.4f}")
```

```
Iteración 0: Coste = 0.0004
Iteración 1: Coste = 0.0124
Iteración 2: Coste = 0.0177
Iteración 3: Coste = 0.0070
Iteración 4: Coste = 0.0000
Iteración 5: Coste = 0.0077
Iteración 6: Coste = 0.0100
Iteración 7: Coste = 0.0040
Iteración 8: Coste = 0.0000
Iteración 9: Coste = 0.0021
Iteración 10: Coste = 0.0054
Iteración 11: Coste = 0.0053
Iteración 12: Coste = 0.0024
Iteración 13: Coste = 0.0001
Iteración 14: Coste = 0.0007
Iteración 15: Coste = 0.0028
Iteración 16: Coste = 0.0032
Iteración 17: Coste = 0.0017
Iteración 18: Coste = 0.0002
Iteración 19: Coste = 0.0003
```

In conclusion.

The implemented Variational Quantum Circuit demonstrates the feasibility of using hybrid architectures to approximate complex functions through classical quantum parameter optimization. The modularity of the presented ansatz allows this design to serve not only as a versatile starting point for general regression tasks but also as a scalable framework for solving highly complex physical problems, such as the evaluation of integral kernels in the Bethe-Salpeter Equation. By integrating the expressive power of entanglement layers with precise classical control, this model achieves an efficient balance between computational complexity and predictive accuracy, establishing itself as a robust and adaptable tool for research in applied quantum computing.